

O CDC-P como um ASIC por Software

Cascatas de Flip-Flops Reconfiguráveis e Aceleração Dinâmica de Clock
em um Kernel Cooperativo Determinístico

Marcos Roberto Braga

Instituto Informal de Educação, Ciência e Tecnologia
noelmrbr_tec@yahoo.com

Pedro Henrique Cerqueira Braga

Colaborador

12 de setembro de 2026

Resumo

Este trabalho propõe uma equivalência conceitual entre o kernel CDC-P (Concurrency Deterministic Control with Preemption) e um ASIC síncrono (Application-Specific Integrated Circuit). Argumentamos que as tarefas do CDC-P se comportam como flip-flops, o tick se comporta como um clock, e o despachador se comporta como lógica combinacional. No entanto, ao contrário de um ASIC físico, o CDC-P exibe cinco capacidades que um ASIC não possui: (i) reconfiguração em tempo de execução da cascata de flip-flops (reordenação de fila via URG-S); (ii) aceleração dinâmica do clock sob carga (ajuste de tick e modo turbo); (iii) auto-regulação temporal por tarefa (homeostase); (iv) isolamento arquitetural de falhas (Xerife, Síndico, pane permanente); e (v) portabilidade radical (mesmo kernel em cinco arquiteturas). Chamamos esse modelo de *ASIC por Software*: ele preserva o determinismo de hardware síncrono enquanto oferece a flexibilidade de software. Resultados empíricos em PIC16, PIC18, AVR, RISC-V e ARM Cortex-M demonstram a viabilidade do modelo.

Palavras-chave: ASIC, CDC-P, hardware síncrono, lógica reconfigurável, clock dinâmico, kernel determinístico, sistemas embarcados.

1 Introdução

Em projeto digital síncrono, um ASIC (Application-Specific Integrated Circuit) é um dispositivo de hardware no qual uma cascata fixa de flip-flops é sincronizada por um único clock. O comportamento do circuito é determinístico: a cada borda de clock, os flip-flops capturam seu próximo estado de acordo com a lógica combinacional. Esse determinismo é uma propriedade física do silício.

Neste trabalho, propomos que o kernel CDC-P exibe o mesmo comportamento determinístico de um ASIC síncrono, mas com uma diferença crucial: **o CDC-P pode reconfigurar sua própria cascata de flip-flops e acelerar seu próprio clock em tempo de execução**. Essa capacidade não existe em um ASIC físico.

Argumentamos que o CDC-P é, de fato, um *ASIC por Software*: um kernel que se comporta como hardware síncrono, mantendo a flexibilidade do software.

1.1 Contribuições

- Uma equivalência formal entre tarefas do CDC-P e flip-flops;
- Uma equivalência formal entre o tick e um clock global;
- Uma análise de cinco capacidades que o CDC-P tem e os ASICs não;

- Um arcabouço de projeto para construir sistemas determinísticos reconfiguráveis por software;
- Validação empírica em cinco arquiteturas.

2 O Modelo ASIC

2.1 Hardware Síncrono

Um ASIC é caracterizado por:

- Um conjunto de **flip-flops** que armazenam estado;
- Um **clock global** que sincroniza todos os flip-flops;
- **Lógica combinacional** que calcula o próximo estado;
- Um **caminho de dados fixo** definido na fabricação;
- **Comportamento determinístico** garantido pela física.

A cada borda de clock, cada flip-flop captura o valor produzido pela lógica combinacional. O resultado é determinístico, reproduzível e independente de eventos externos (exceto pelas entradas).

2.2 Propriedades de um ASIC Síncrono

Tabela 1: Propriedades de um ASIC síncrono

Propriedade	Descrição
Determinismo	Garantido pela física
Clock	Fixo pelo oscilador
Caminho de dados	Fixo na fabricação
Reconfiguração	Impossível
Isolamento de falhas	Não suportado
Portabilidade	Específico da aplicação

3 O CDC-P como um ASIC por Software

3.1 Equivalência Estrutural

Tabela 2: Equivalência estrutural entre ASIC e CDC-P

ASIC	CDC-P
Flip-flops	Tarefas (funções que retornam)
Clock global	Tick (timer ou SysTick)
Lógica combinacional	Despachador
Caminho de dados	Fila de prioridades
Fabricação	Configuração do kernel

A cada tick, cada tarefa é avaliada pelo despachador de acordo com aritmética modular:

$$\text{executa tarefa } i \iff (\text{tick} - \text{timer}_i) \geq \text{period}_i \quad (1)$$

Isso é matematicamente determinístico, assim como o próximo estado de um flip-flop é determinístico em hardware.

3.2 O que o CDC-P tem que um ASIC não tem

3.2.1 Reconfiguração em Tempo de Execução da Cascata de Flip-Flops

Em um ASIC, a cascata de flip-flops é fixa na fabricação. No CDC-P, a fila pode ser reordenada em tempo de execução através do URG-S (Urgência Situacional):

```
set_urg_task2(); // Reordena a execução na próxima varredura
```

Isso permite que uma tarefa *fure a fila* sem interromper outras tarefas. O efeito é equivalente a reconfigurar o caminho de dados de um ASIC em tempo real.

3.2.2 Aceleração Dinâmica do Clock

Em um ASIC, o clock é fixo pelo oscilador. No CDC-P, o tick pode ser ajustado em tempo de execução:

Tabela 3: Modos dinâmicos de tick no CDC-P

Modo	Tick	Uso
Turbo	1 ms	Controle crítico
Rápido	2 ms	Monitoramento intensivo
Normal	8 ms	Uso geral
Lento	50 ms	Logging de baixo consumo

O kernel pode alternar modos automaticamente com base na carga (Task8 — Acelerador) ou por solicitação da aplicação (comando 'd').

3.2.3 Auto-Regulagem Temporal por Tarefa

Em um ASIC, nenhum elemento pode se desacelerar. No CDC-P, cada tarefa se auto-regula:

```
timer_ex_task1 = (tick - timer_task1) / task1_periodo;
if(timer_ex_task1) { task1_periodo = task1_periodo + 1; }
```

Isso é homeostase: a tarefa desacelera sob carga, prevenindo o colapso do sistema.

3.2.4 Isolamento Arquitetural de Falhas

Um ASIC não pode isolar um flip-flop defeituoso. O CDC-P isola tarefas defeituosas através de:

- **Xerife (Task10):** detecta falha crônica e isola a tarefa permanentemente;
- **Síndico (Task9):** recupera tarefas bloqueadas;
- **Fail-safe:** reseta o sistema se o Síndico falhar;
- **Pane permanente:** bloqueia uma tarefa até intervenção externa.

Isso é equivalente a reconfigurar o ASIC para contornar uma região defeituosa.

3.2.5 Portabilidade Radical

Um ASIC é específico da aplicação. O CDC-P é portátil entre:

- PIC16, PIC18 (8 bits);
- AVR AT90S8515 (8 bits);
- RISC-V CH32V003 (32 bits);
- ARM Cortex-M3 STM32F103 (32 bits).

O mesmo código de kernel roda em todas as plataformas.

4 Validação Empírica

4.1 Plataformas

Tabela 4: Plataformas testadas

MCU	Arquitetura	Clock	RAM	Flash
PIC16F628A	8 bits	4 MHz	224 B	2 KB
PIC18F6722	8 bits	8 MHz	4 KB	128 KB
AT90S8515	AVR 8 bits	8 MHz	512 B	8 KB
CH32V003F4P6	RISC-V	48 MHz	2 KB	16 KB
STM32F103	ARM Cortex-M3	72 MHz	20 KB	64 KB

4.2 Determinismo

Em todas as plataformas, o contador de segundos permaneceu exato sob carga. Sem congelamento, sem saltos. Isso confirma a equivalência com hardware síncrono.

4.3 Reconfiguração

O URG-S foi testado a 1000 eventos/s. O sistema permaneceu estável, e as tarefas urgentes executaram na próxima varredura do despachador, com latência de 25–112 μ s.

4.4 Clock Dinâmico

O tick foi alternado entre 1, 2, 8 e 50 ms em tempo de execução. Nenhum glitch foi observado; o sistema se adaptou suavemente.

4.5 Isolamento de Falhas

Os mecanismos Xerife e Síndico isolaram tarefas deliberadamente defeituosas sem afetar as saudáveis.

5 Discussão

5.1 Por que isso importa

ASICs são rápidos e determinísticos, mas inflexíveis. Software é flexível, mas frequentemente não-determinístico. O CDC-P oferece o melhor dos dois mundos:

- Determinismo de hardware síncrono;
- Flexibilidade de software;
- Reconfiguração em tempo de execução;
- Portabilidade entre arquiteturas.

5.2 Onde o CDC-P se destaca

- Sistemas de tempo real crítico com tarefas periódicas;
- Sistemas embarcados com restrições de recursos;
- Sistemas que exigem tolerância a falhas;
- Sistemas que exigem adaptação dinâmica.

5.3 Onde os ASICs são superiores

- Desempenho extremo (GHz);
- Eficiência energética extrema (pJ/op);
- Paralelismo massivo (milhares de caminhos paralelos);
- Volumes de produção (milhões de unidades).

As duas abordagens são complementares, não concorrentes.

6 Conclusão

Propomos uma equivalência conceitual entre o CDC-P e um ASIC síncrono, e identificamos cinco capacidades que o CDC-P tem e os ASICs não: (i) reconfiguração em tempo de execução da cascata de flip-flops; (ii) aceleração dinâmica do clock; (iii) auto-regulagem temporal por tarefa; (iv) isolamento arquitetural de falhas; e (v) portabilidade radical.

Chamamos esse modelo de *ASIC por Software*: um kernel que se comporta como hardware síncrono, mantendo a flexibilidade do software.

“O CDC-P é um ASIC que pode reescrever seu próprio silício em tempo de execução.”

Agradecimentos

O autor agradece a Pedro Henrique Cerqueira Braga pela colaboração no desenvolvimento e testes do CDC-P.

Referências

- [1] LIU, C. L.; LAYLAND, J. W. Scheduling Algorithms for Multiprogramming in a Hard-Real-Time Environment. *Journal of the ACM*, v. 20, n. 1, p. 46–61, 1973.
- [2] DIJKSTRA, E. W. Go To Statement Considered Harmful. *Communications of the ACM*, v. 11, n. 3, p. 147–148, 1968.
- [3] DIJKSTRA, E. W. The Humble Programmer. *Communications of the ACM*, v. 15, n. 10, p. 859–866, 1972.

- [4] WESTE, N.; HARRIS, D. *CMOS VLSI Design: A Circuits and Systems Perspective*. 4th ed. Addison-Wesley, 2010.
- [5] RABAEY, J.; CHANDRAKASAN, A.; NIKOLIC, B. *Digital Integrated Circuits: A Design Perspective*. 2nd ed. Prentice Hall, 2003.
- [6] FreeRTOS Documentation. <https://www.freertos.org/>. Acesso em: 2026.
- [7] BARR, M. *Programming Embedded Systems in C and C++*. O'Reilly, 1999.
- [8] LABROSSE, J. J. *$\mu C/OS-III$: The Real-Time Kernel*. Micrium, 2010.
- [9] JONES, M. What Really Happened on Mars? *NASA JPL*, 1997.
- [10] ASTIER, J. Y. Multiple FSMs vs RTOS. *LinkedIn*, 2024.